

Manual Técnico-Conceptual — Pipeline ETL Pokémon

Propósito de este documento: Entender a fondo cada fase del pipeline, los conceptos técnicos detrás de cada decisión, y cómo explicar el proyecto en una entrevista de Data Analyst / Data Engineer.

Índice

- ¿Qué es ETL y por qué importa?
- Arquitectura general del pipeline
- Fase Extract — Consumir la PokémonAPI
- Fase Transform — Limpiar y enriquecer datos
- Fase Load — Persistir y visualizar
- Infraestructura de soporte
- Testing con pytest
- El dataset resultante
- Guía de entrevista — Cómo explicar este proyecto
- Glosario de términos clave

1. ¿Qué es ETL y por qué importa?

Definición

ETL son las siglas de **Extract, Transform, Load** — los tres pasos universales para mover datos desde una fuente hasta un destino donde puedan ser analizados.

Paso	Qué hace	Analogía
Extract	Obtiene los datos crudos de la fuente	Recoger ingredientes del mercado
	original	
Transform	Limpia, normaliza y enriquece los datos	Preparar y cocinar los ingredientes
Load	Carga el dato limpio al destino final	Servir el plato en la mesa

¿Por qué no leer directo de la fuente cada vez?

Conectar directamente un dashboard a una API pública tiene tres problemas:

- **Velocidad** — Cada carga de página haría decenas de requests HTTP. Una consulta SQL sobre un CSV/SQLite es 100x más rápida.
- **Disponibilidad** — Si la API falla o cambia, el dashboard se cae. Con ETL, los datos están en local y el dashboard no depende de que la API esté disponible.
- **Reproducibilidad** — Los datos de la API pueden cambiar. El JSON crudo guardado en `data/raw/` es el estado exacto en el momento de la extracción, lo que permite re-procesar sin volver a consumir la API.

ETL vs ELT

	ETL	ELT
Orden	Transformar antes de cargar	Cargar crudo, transformar dentro del destino
Cuándo	Fuentes heterogéneas, destinos relacionales	Warehouses modernos (BigQuery, Snowflake)
Este proyecto	ETL — transformamos en Python antes de cargar a SQLite	—

2. Arquitectura general del pipeline

Estructura de módulos

```

src/
├─ config.py           ← Rutas, constantes, variables de entorno
├─ extract/
│   └─ api_client.py   ← Cliente HTTP + retry (Fase E)
├─ transform/
│   └─ cleaner.py       ← Flattening + métricas derivadas (Fase T)
└─ load/
    └─ exporter.py      ← SQLite + visualizaciones (Fase L)

main.py                ← Orquestador: lee args y llama a cada fase

```

Flujo de datos

```

PokéAPI
├─ HTTP GET /pokemon/{name}  (×151 requests)
▼
data/raw/pokemon_raw.json    ← JSON crudo guardado tal como viene
├─ flatten_pokemon()
▼
data/processed/pokemon_clean.csv ← 151 filas × 21 columnas, plano
├─ load_to_sqlite()
▼
data/output/pokemon.db       ← Base de datos SQLite (tabla: pokemon)
├─                               ← dashboard.html (Plotly interactivo)
└─ generate_viz()            ← stats_report.png (Matplotlib estático)

```

El principio de fases independientes

Cada fase puede ejecutarse por separado gracias al argumento `--phase`:

```

python main.py --phase extract  # Solo descarga el JSON
python main.py --phase transform # Lee el JSON, produce el CSV
python main.py --phase load     # Lee el CSV, exporta a SQLite y gráficas
python main.py --limit 151      # Ejecuta las tres fases en secuencia

```

Esto es una práctica real de data engineering: si cambia la lógica de transformación, re-procesamos sin volver a

consumir la API.

3. Fase Extract — Consumir la PokéAPI

Archivo: `src/extract/api_client.py`

¿Qué es una REST API?

Una **REST API** expone recursos a través de URLs (endpoints) y el protocolo HTTP. Se interactúa con ella enviando **requests** HTTP y recibiendo respuestas en **JSON**.

La PokéAPI es pública, sin autenticación, con un límite de ~100 requests/minuto.

Endpoints utilizados

Endpoint	Qué devuelve
<code>`GET /pokemon?limit=151&offset=0`</code>	Lista de 151 Pokémon con nombre y URL de detalle
<code>`GET /pokemon/{name}`</code>	Objeto completo de un Pokémon: tipos, stats, habilidades, altura, peso, etc.

PokeAPIClient — El cliente HTTP

```
class PokeAPIClient:
    def __init__(self):
        self.session = requests.Session()
        self.session.headers.update({"Accept": "application/json"})
```

Se usa `requests.Session()` en lugar de `requests.get()` directamente porque:

- **Reutiliza la conexión TCP (keep-alive)** — menos latencia en 151 requests consecutivos
- **Headers compartidos** — se definen una vez, no en cada llamada

Rate limiting y el `time.sleep()`

La PokéAPI tiene un límite de ~100 requests/minuto. Sin control, 151 requests seguidos podrían generar errores 429 (Too Many Requests).

```
for entry in pokemon_list:
    detail = self.fetch_pokemon_detail(entry["name"])
    all_pokemon.append(detail)
    time.sleep(self.delay) # 0.3s entre requests → ~3.3 req/seg → bien bajo el límite
```

`REQUEST_DELAY = 0.3` viene de la variable de entorno `REQUEST_DELAY` en `.env`. Esto permite ajustarlo sin tocar el código.

Tenacity — Reintentos automáticos con backoff exponencial

Las APIs externas fallan. Una conexión se puede cortar, un servidor puede estar temporalmente sobrecargado. Sin reintentos, un error puntual en el request #147 arruinaría toda la extracción.

```
@retry(
    stop=stop_after_attempt(MAX_RETRIES), # Máximo 3 intentos
    wait=wait_exponential(multiplier=1, min=2, max=10), # Espera: 2s, 4s, 8s
    retry=retry_if_exception_type(requests.RequestException),
    before_sleep=lambda retry_state: logger.warning(...),
```

```
)
def _get(self, endpoint: str) -> dict:
    ...
```

Backoff exponencial: En lugar de reintentar inmediatamente (lo que podría saturar un servidor ya sobrecargado), se espera el doble cada vez: 2s → 4s → 8s. Es una práctica estándar para consumir APIs externas de forma respetuosa.

Persistencia del JSON crudo

```
with open(RAW_JSON_PATH, "w", encoding="utf-8") as f:
    json.dump(pokemon_data, f, ensure_ascii=False, indent=2)
```

¿Por qué guardar el crudo antes de transformar?

- Si la transformación falla (bug en el código), no hay que volver a consumir la API
- Es el "source of truth" — si alguien cuestiona los datos, se puede verificar contra el JSON original
- Permite probar distintas estrategias de transformación sobre el mismo conjunto de datos

tqdm — Barra de progreso

```
for entry in tqdm(pokemon_list, desc="Extrayendo", unit="pokemon"):
```

tqdm (del árabe "taqaddum" = progreso) muestra una barra en consola que indica velocidad, tiempo restante y progreso actual. Para procesos de 151 requests (30-60 segundos), la retroalimentación visual es esencial.

4. Fase Transform — Limpiar y enriquecer datos

Archivo: src/transform/cleaner.py

El problema del JSON anidado

La PokéAPI devuelve un objeto JSON profundamente anidado. Un DataFrame de pandas necesita datos planos (una fila = un registro, una columna = un campo).

JSON crudo (estructura anidada):

```
{
  "id": 1,
  "name": "bulbasaur",
  "types": [
    {"slot": 1, "type": {"name": "grass"}},
    {"slot": 2, "type": {"name": "poison"}}
  ],
  "stats": [
    {"base_stat": 45, "stat": {"name": "hp"}},
    {"base_stat": 49, "stat": {"name": "attack"}}
  ],
  "abilities": [
    {"ability": {"name": "overgrow"}},
    {"ability": {"name": "chlorophyll"}}
  ]
}
```

Registro plano (lo que necesitamos):

id	name	type_primary	type_secondary	hp	attack	abilities
1	bulbasaur	grass	poison	45	49	overgrow, chlorophyll

flatten_pokemon() — Paso a paso

```
def flatten_pokemon(raw: dict) -> dict:
    type_primary, type_secondary = _extract_types(raw["types"])
    stats = _extract_stats(raw["stats"])
    abilities = _extract_abilities(raw["abilities"])
    bst = sum(stats.values())
    return {"id": ..., "type_primary": type_primary, ..., **stats, "bst": bst}
```

`_extract_types`: Ordena por slot (1 = primario, 2 = secundario) y extrae los nombres.

`_extract_stats`: Convierte la lista de stats a un diccionario plano. Normaliza el nombre de la clave: `special-attack` → `special_attack` (snake_case, compatible con Python y SQL).

`_extract_abilities`: Extrae los nombres y los une como string separado por coma.

Campos derivados calculados

Los "campos derivados" no existen en la API — son métricas que construimos con los datos que tenemos. Son lo que demuestra criterio analítico.

Campo	Fórmula	Por qué es útil
<code>`bst`</code>	<code>`hp + attack + defense + special_attack + special_defense + speed`</code>	Métrica estándar del universo Pokémon para comparar poder total
<code>`height_m`</code>	<code>`height_dm / 10`</code>	La API da altura en decímetros. Los metros son más intuitivos
<code>`weight_kg`</code>	<code>`weight_hg / 10`</code>	La API da peso en hectogramos. Los kilos son más intuitivos
<code>`bmi_index`</code>	<code>`weight_kg / height_m²`</code>	Índice de masa corporal: cuantifica la "densidad" del Pokémon
<code>`generation`</code>	Lookup por rango de ID	IDs 1-151 = Gen I, 152-251 = Gen II, etc.
<code>`power_tier`</code>	<code>`pd.cut()`</code> sobre BST	Categorización discreta del nivel de poder
<code>`sprite_url`</code>	URL construida con el ID	Enlace directo a la imagen oficial sin llamar a la API

pd.cut() — Categorización por rangos

```
df["power_tier"] = pd.cut(
    df["bst"],
    bins=[0, 299, 399, 499, 599, 9999],
    labels=["Débil", "Bajo", "Medio", "Alto", "Legendario-tier"],
)
```

`pd.cut()` divide una variable continua en intervalos discretos. Convierte el BST (número del 0 al 9999) en una categoría legible. El resultado es de tipo `category` — más eficiente en memoria que `str` para variables con pocos valores únicos.

Limpieza de datos

```
df.drop_duplicates(subset=["id"], inplace=True) # Por si la API devolvió el mismo ID dos veces
df.dropna(subset=["name", "type_primary"], inplace=True) # Un Pokémon sin nombre o tipo es inútil
```

Solo eliminamos registros donde faltan campos **críticos**. El campo `type_secondary` puede ser `NaN` perfectamente (un Pokémon puede ser de un solo tipo).

Convenciones de naming: snake_case

Los nombres de columnas usan `snake_case`: palabras en minúsculas separadas por guiones bajos. Esto es compatible con:

- Python (variables)
- SQL (nombres de columna)
- pandas (acceso por atributo: `df.type_primary`)

5. Fase Load — Persistir y visualizar

Archivo: `src/load/exporter.py`

SQLite — Base de datos sin servidor

```
with sqlite3.connect(SQLITE_DB_PATH) as conn:
    df.to_sql("pokemon", conn, if_exists="replace", index=False)
```

¿Por qué SQLite y no PostgreSQL o MySQL?

- No requiere instalar ni configurar un servidor
- El archivo `.db` es portable — se abre en cualquier cliente SQL (DBeaver, TablePlus, DB Browser)
- Para 151 registros, SQLite es más que suficiente
- Para un portafolio, eliminar dependencias externas baja la fricción de demostración

`if_exists="replace"`: Si la tabla ya existe, la reemplaza completamente. Alternativas: `"append"` (agregar filas) o `"fail"` (lanzar error si existe).

`index=False`: No incluye el índice de pandas como columna en SQL. El índice es un artefacto de pandas, no es parte del dato.

Visualizaciones — Plotly vs Matplotlib/Seaborn

El proyecto genera dos tipos de visualizaciones:

	Plotly	Matplotlib + Seaborn
Resultado	<code>`dashboard.html`</code>	<code>`stats_report.png`</code>
Interacción	Hover, zoom, filtros	Estática
Uso	Dashboard web	Reporte/informe
Cuándo usar	Presentaciones, dashboards	PDFs, informes, publicaciones

`make_subplots()`: Plotly permite organizar múltiples gráficas en una cuadrícula (`rows × cols`). Cada gráfica se añade con `fig.add_trace(..., row=X, col=Y)`.

Paleta de colores por tipo: `TYPE_COLORS` mapea cada tipo Pokémon a su color canónico del juego. Esto no es decorativo — da contexto semántico: el color azul de `"water"` es inmediatamente reconocible.

6. Infraestructura de soporte

config.py — Fuente única de verdad para configuración

```
BASE_DIR = Path(__file__).parent.parent # Raíz del proyecto

RAW_JSON_PATH = DATA_RAW_DIR / "pokemon_raw.json"
PROCESSED_CSV_PATH = DATA_PROCESSED_DIR / "pokemon_clean.csv"
SQLITE_DB_PATH = DATA_OUTPUT_DIR / "pokemon.db"
```

¿Por qué centralizar rutas? Si `data/raw/` se renombra, el cambio se hace en un solo lugar (`config.py`) y todos los módulos que lo importan lo reciben automáticamente. Sin `config.py`, cada módulo tendría la ruta hardcodeada.

`Path` de `pathlib` vs strings: `Path("data/raw/pokemon_raw.json")` es multiplataforma — funciona en Windows (`\`) y macOS/Linux (`/`). Los strings hardcodeados con `/` fallan en Windows.

Variables de entorno con python-dotenv

```
# .env (nunca en git)
POKEMON_LIMIT=151
REQUEST_DELAY=0.3
MAX_RETRIES=3

# config.py
from dotenv import load_dotenv
load_dotenv()
POKEMON_LIMIT = int(os.getenv("POKEMON_LIMIT", 151)) # 151 es el valor por defecto
```

¿Por qué `.env` y no hardcodear los valores?

- Un colaborador puede ajustar parámetros sin tocar el código
- Si el proyecto tuviera una API key real, nunca iría en el código (podría quedar en git)
- `.env.example` sirve como documentación: "estos son los parámetros que el proyecto acepta"

Logging con Loguru

```
logger.info("Obteniendo lista de 151 Pokémon...")
logger.success("Extracción completa: 151 Pokémon descargados.")
logger.error(f"Error al obtener {entry['name']}: {exc}")
```

Loguru es una alternativa moderna al módulo `logging` estándar de Python. Ventajas:

- Sintaxis más simple (no necesita `getLogger()`, `handlers`, `formatters`)
- Niveles de color en consola
- Rotación de archivos automática (`rotation="10 MB"`)
- Incluye nombre de módulo y línea en los logs

Logs en archivo: Cada ejecución crea un archivo en `logs/etl_YYYYMMDD_HHMMSS.log` con nivel `DEBUG`. La consola muestra solo `INFO` y superior.

main.py — CLI con argparse

```
parser.add_argument("--phase", choices=["extract", "transform", "load", "all"])
parser.add_argument("--limit", type=int, default=POKEMON_LIMIT)
parser.add_argument("--no-viz", action="store_true")
```

`argparse` convierte argumentos de línea de comandos en variables Python con validación automática. `--phase extract` falla con un mensaje útil si el valor no está en `choices`.

Importaciones diferidas: Los módulos de cada fase se importan solo cuando se necesitan:

```
if args.phase in ("extract", "all"):
    from src.extract.api_client import run_extract    # ← Importado aquí, no al inicio
```

Esto evita cargar dependencias de una fase que no se va a ejecutar.

7. Testing con pytest

Archivos: tests/conftest.py, tests/test_extract.py, tests/test_transform.py

¿Qué son los tests unitarios?

Un **test unitario** verifica que una función específica produce el resultado correcto dado un input controlado. No llama a servicios externos (APIs, bases de datos reales).

conftest.py — Fixtures compartidos

```
@pytest.fixture
def raw_pokemon():
    return MOCK_POKEMON_RAW.copy()

@pytest.fixture
def clean_dataframe():
    from src.transform.cleaner import flatten_pokemon
    record = flatten_pokemon(MOCK_POKEMON_RAW)
    return pd.DataFrame([record])
```

Un **fixture** es un dato de prueba que pytest prepara antes de cada test. Al declararlo en `conftest.py`, está disponible para todos los módulos de test sin necesidad de importarlo.

¿Por qué usar datos mock y no llamar a la API real?

- **Velocidad** — Los tests deben correr en <1 segundo. 151 requests HTTP toman 30-60 segundos.
- **Determinismo** — Los tests deben producir siempre el mismo resultado. Si la API cambia (agrega un campo, cambia un valor), los tests no deberían fallar.
- **Sin dependencia de red** — Los tests deben pasar en cualquier entorno (CI/CD, sin internet).

Ejecutar los tests

```
pytest tests/ -v          # Verbose: muestra nombre de cada test
pytest tests/ --tb=short  # Solo muestra la línea que falló
```

8. El dataset resultante

Dimensiones

151 filas (un Pokémon por fila) × **21 columnas**

Descripción de columnas

Columna	Tipo	Fuente	Descripción
`id`	int	API	ID oficial de la PokéAPI (1-151 para Gen I)
`name`	str	API	Nombre en minúsculas (bulbasaur, ivysaur...)
`type_primary`	str	API	Tipo principal (grass, fire, water...)
`type_secondary`	str / NaN	API	Tipo secundario, NaN si es monotipo
`hp`	int	API	Puntos de vida base
`attack`	int	API	Ataque físico base
`defense`	int	API	Defensa física base
`special_attack`	int	API	Ataque especial base
`special_defense`	int	API	Defensa especial base
`speed`	int	API	Velocidad base
`bst`	int	**Derivado**	Base Stat Total: suma de las 6 stats
`generation`	int	**Derivado**	Generación por rango de ID (1 para Gen I)
`height_m`	float	**Derivado**	Altura en metros (API da decímetros)
`weight_kg`	float	**Derivado**	Peso en kilogramos (API da hectogramos)
`bmi_index`	float	**Derivado**	Índice peso/altura ²
`power_tier`	category	**Derivado**	Débil / Bajo / Medio / Alto / Legendario-tier
`abilities`	str	API	Habilidades separadas por coma
`base_experience`	int	API	Experiencia base al ser derrotado
`height_dm`	int	API	Altura original en decímetros (antes de conversión)
`weight_hg`	int	API	Peso original en hectogramos (antes de conversión)
`sprite_url`	str	**Derivado**	URL a la imagen del Pokémon
`is_legendary`	NaN	—	Placeholder (no disponible en este endpoint de la API)

Calidad del dato

- **Nulos en `type\_secondary`:** ~56% de los Pokémon son monotipo. Esto es dato válido, no error.
- **Nulos en `is\_legendary`:** El endpoint `/pokemon` no incluye esta info. Requeriría el endpoint `/pokemon-species` (fuera del scope).

- **Duplicados:** 0 (garantizado por `drop_duplicates(subset=["id"])`).

9. Guía de entrevista — Cómo explicar este proyecto

Pitch de 60 segundos

"Construí un pipeline ETL completo en Python que consume la PokéAPI pública, descarga 151 registros con sus estadísticas, tipos y habilidades, los transforma calculando métricas derivadas como el Base Stat Total y el índice de masa corporal, y los carga a una base de datos SQLite. El pipeline genera visualizaciones interactivas con Plotly y una app web con Streamlit que permite explorar y comparar Pokémon. Usé tenacity para manejar reintentos automáticos, loguru para logging estructurado, y pytest para tests unitarios. El proyecto demuestra que puedo construir un flujo de datos completo desde la fuente hasta la visualización."

Las 3 decisiones de diseño y sus justificaciones

1. ¿Por qué guardar el JSON crudo antes de transformar?

"Es una práctica de data engineering real: separar el raw del procesado permite re-ejecutar la transformación con nueva lógica sin volver a consumir la API. Si la lógica de power_tier cambia, simplemente corro `python main.py --phase transform` sobre el JSON que ya tengo."

2. ¿Por qué SQLite y no PostgreSQL?

"Es un proyecto de portafolio local. SQLite elimina dependencias externas: cualquiera que clone el repo puede ejecutar el pipeline y abrir `pokemon.db` con cualquier cliente SQL sin instalar nada. Para un caso de producción con usuarios concurrentes usaría PostgreSQL."

3. ¿Por qué Streamlit y no una app web tradicional?

"Para un perfil de Data Analyst, Streamlit es la herramienta correcta. Es Python puro, el ecosistema de datos (pandas, plotly) se integra de forma nativa, y permite construir dashboards interactivos sin salir del dominio. Node.js tiene sentido en perfiles full-stack, pero para DA demuestra las habilidades equivocadas."

Preguntas frecuentes en entrevista

¿Qué harías diferente si esto fuera producción?

"Reemplazaría SQLite con PostgreSQL o Redshift, usaría Airflow o Prefect para orquestar el pipeline con scheduling, añadiría alertas cuando el pipeline falla, y configuraría un sistema de versionado de datos (como Delta Lake). También añadiría validación de esquema con Great Expectations o pandera."

¿Cómo escalarías esto a 1000 Pokémon?

"El código ya lo soporta: `--limit 1000` funcionaría. Para escala real, usaría procesamiento paralelo con `asyncio` + `httpx` para las requests HTTP concurrentes, respetando el rate limit con un semáforo."

¿Qué demostraría este proyecto en tu CV?

"Que conozco el flujo completo del dato: desde la fuente (API), pasando por limpieza y enriquecimiento (pandas), hasta la visualización (Plotly + Streamlit). También que conozco buenas prácticas: separación de responsabilidades, variables de entorno, logging, testing."

10. Glosario de términos clave

Término	Definición rápida
API	Interfaz de programación que permite a dos sistemas intercambiar datos
REST API	API que usa HTTP como protocolo y URLs para identificar recursos
JSON	JavaScript Object Notation — formato de texto estándar para datos estructurados
Rate limiting	Restricción del número de requests que un cliente puede hacer en un periodo
Backoff exponencial	Estrategia de reintento donde el tiempo de espera se duplica en cada intento
Flattening	Proceso de convertir una estructura de datos anidada en una tabla plana
snake_case	Convención de naming: palabras en minúsculas separadas por guiones bajos
DataFrame	Estructura de datos tabular de pandas (filas × columnas)
BST	Base Stat Total — suma de las 6 estadísticas base de un Pokémon
pd.cut()	Función de pandas para convertir una variable continua en categorías discretas
SQLite	Base de datos relacional que vive en un solo archivo, sin servidor
to_sql()	Método de pandas para exportar un DataFrame a una tabla SQL
Fixture (pytest)	Dato de prueba preparado antes de ejecutar un test
Mock data	Datos falsos pero realistas usados en tests para evitar dependencias externas
loguru	Librería de logging para Python con sintaxis simplificada
tenacity	Librería Python para implementar reintentos automáticos con decoradores
tqdm	Librería para mostrar barras de progreso en bucles Python
argparse	Módulo estándar de Python para parsear argumentos de línea de comandos
pathlib.Path	Clase Python para manejar rutas de archivos de forma multiplataforma
dotenv	Librería para cargar variables de entorno desde un archivo <code>`.env`</code>